Problem Challenge 2: Scheduling Tasks (hard)

Problem Statement

You are given a list of tasks that need to be run, in any order, on a server. Each task will take one CPU interval to execute but once a task has finished, it has a cooling period during which it can't be run again. If the cooling period for all tasks is 'K' intervals, find the minimum number of CPU intervals that the server needs to finish all tasks.

If at any time the server can't execute any task then it must stay idle.

Example 1:

```
Input: [a, a, a, b, c, c], K=2
Output: 7
Explanation: a -> c -> b -> a -> c -> idle -> a
```

Example 2:

```
Input: [a, b, a], K=3
Output: 5
Explanation: a -> b -> idle -> idle -> a
```

Try it yourself

Try solving this question here:

Java

```java
import java.util.*;

class Solution {

  public int scheduleTasks(char[] tasks, int k) {
    int intervalCount = 0;
    // TODO: Write your code here
    return intervalCount;
  }

}

```

Test Case | Run Code Result | Stdout

```
["a","a","a","b","c","c"]
2
```

Use Example Testcases | Run | Submit

← Previous

Solution: Problem Challenge 1: Rearrange String K Distance Apart

Next →

Solution: Problem Challenge 2: Scheduling Tasks

Mark as Completed